Wyniki etapu IV: Implementacja i testy

EcoRoute

Projektowanie oprogramowania

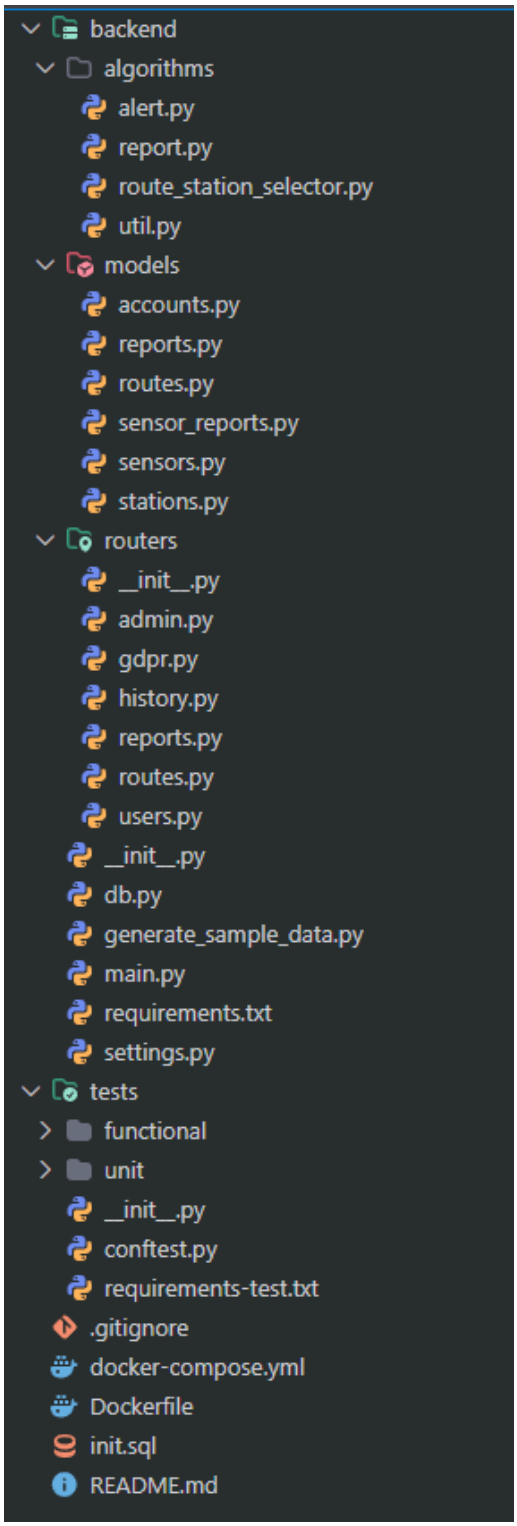
Skład zespołu:
Marek Gagaczowski
Błażej Kowal
Alina Lenart
Michał Strugarek

Prowadzący:
Marcin Jodłowiec

Implementacja

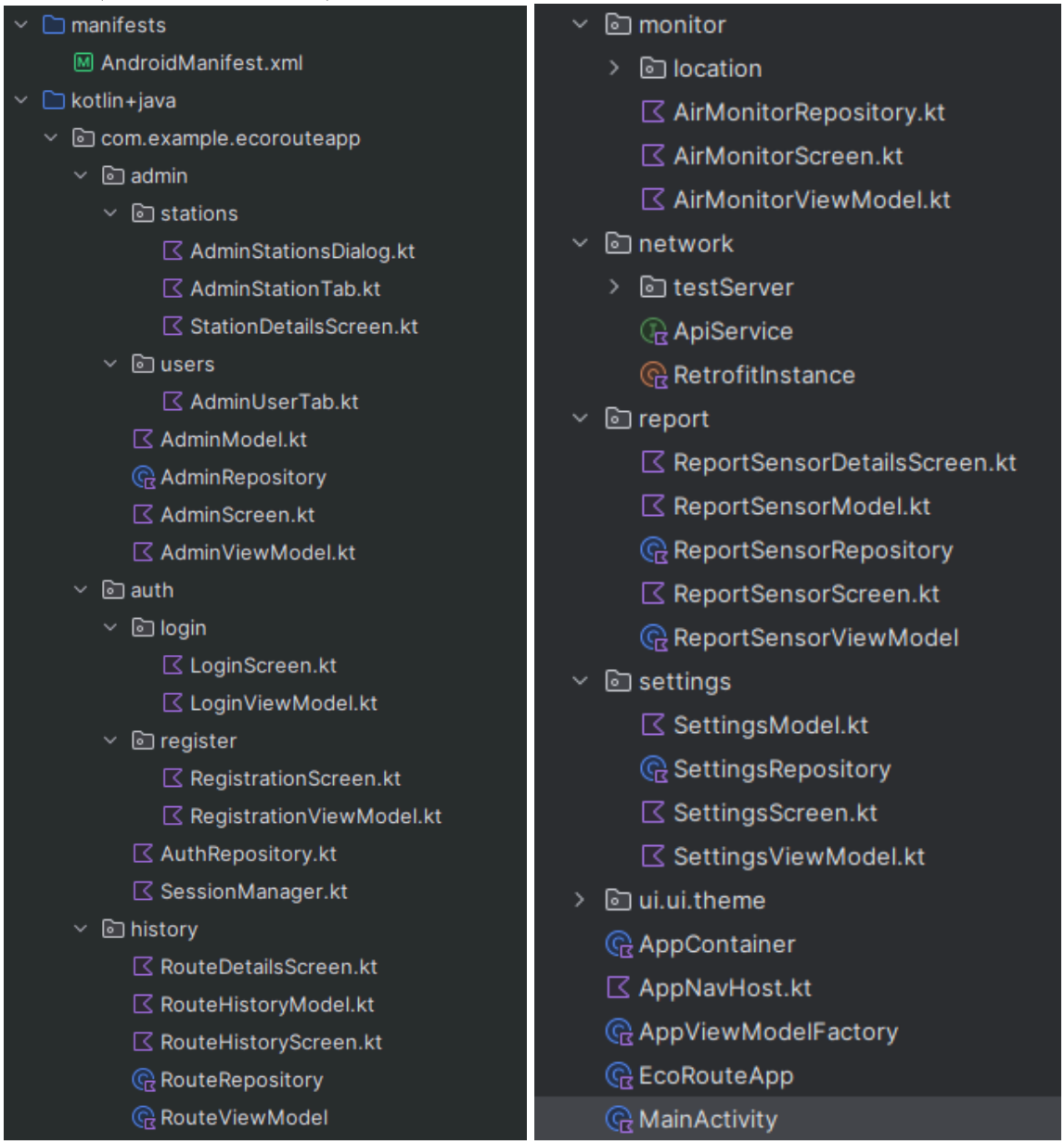
1. Struktura kodu

Backend:



PostresSQL: znajduje się w init.sql

Frontend (Kotlin, AndroidStudio):



2. Wzorce projektowe

Backend: (znajduje się w backend/README.md)
- Router (FastAPI Router Pattern) – Każda domena funkcjonalna (np. użytkownicy, trasy, raporty) posiada osobny router w katalogu backend/routers/, co pozwala na modularną organizację kodu i łatwe zarządzanie endpointami.
- Model (Data Model Pattern) – Struktury danych i logika bazy danych są wydzielone do osobnych plików w katalogu backend/models/, co ułatwia zarządzanie modelem danych i jego rozwojem.
- Util/Helper (Utility Pattern) – Funkcje pomocnicze wykorzystywane w wielu miejscach aplikacji umieszczone są w pliku backend/algorithms/util.py, co pozwala na ich ponowne użycie i ogranicza duplikację kodu.

Frontend:
Wzorzec architektoniczny - MVVM, odpowiednia separacja zależności na każdym poziomie
Kotlin wspiera i upraszcza implementację wzorców projektowych - wykorzystywane w ten sposób były również w tym projekcie. Ważniejsze przykłady:
- fasada - widoczny w połączeniu z MVVM reprezentowany przez repozytoria dla poszczególnych elementów aplikacji
- factory method - zaimplementowany w postaci AppViewModelFactory, który odpowiedzialny jest za tworzenie ViewModeli ze wstrzykiwanymi odpowiednio zależnościami
- singleton - w Kotlinie jest to typ “object”, używany w projekcie pod nazwą RetrofitInstance do łączenia się z RestApi z backendem
- observer/publisher-subscriber - powszechnie wspierany do komunikacji odczytywania stanu ViewModelu przez View, użyty tutaj w postaci Kotlin StateFlow i Flow

3. Dokumentacja kodu

[Info if any source code documentation was generated (try to assess the coverage), and with which tool. Option.]

Testowanie

1. Testy systemowe - przypadki/procedury testowe

1.1 <Monitor Air Quality (User)>

TC ID: TC-001
Test idea: Verify that user can retrieve current air quality data for an active route
Preconditions:
- User is logged in
- Active route exists (ID=1)
- Route has waypoints with coordinates
- Active measurement stations exist in system

Step	Test data	Expected results
Send GET request to /routes/1	route_id = 1	System retrieves latest waypoint
System selects nearest stations	-	2-5 stations selected based on distance
System retrieves sensor readings	-	Latest PM2.5, PM10, AQI values obtained
System calculates averages	-	Average values calculated

Verify response	-	Response code: 200 OK JSON contains: PM25, PM10, AQI (float/int) Values are reasonable (PM25: 0–500, PM10: 0–600, AQI: 0–500)

TC ID: TC-002
Test idea: Verify system behavior when route does not exist
Preconditions:
- User is logged in
- Route with ID=99999 does not exist

Step	**Test data**	**Expected results**
Send GET request to /routes/99999	route_id = 99999	System attempts to find route
System checks database	-	Route not found in database
Verify error response	-	Response code: 404 Not Found JSON contains: {"detail": "Route not found"}

1.2 <Log Into Application (Guest)>

TC ID: TC-003
Test idea: Verify successful login with valid credentials
Preconditions:
- User account exists in system
- Email: test@example.com
- Password: ValidPassword123

Step	**Test data**	**Expected results**
Send POST request to /auth/login	{"email":"test@example.com","password":"ValidPassword123"}	System receives credentials
System verifies email	-	Email found in database
System verifies password	-	Password hash matches (bcrypt)
System generates JWT token	-	Token generated with user_id and email
Verify response	-	Response code: 200 OK JSON contains: accessToken (string), userId (integer) Token is valid JWT format

TC ID: TC-004
Test idea: Verify login rejection with invalid password

Preconditions: - User account exists - Email: test@example.com		
Step	**Test data**	**Expected results**
Send POST request to /auth/login	{"email":"test@example.com","password":"WrongPassword"}	System receives credentials
System verifies email	-	Email found in database
System verifies password	-	Password hash does not match
Verify error response	-	Response code: 401 Unauthorized JSON contains: {"detail": "Invalid credentials"}

1.3 <Manage GDPR (User)>

TC ID: TC-005 **Test idea:** Verify user can retrieve current GDPR settings **Preconditions:** - User is logged in (user_id=1) - GDPR settings exist for user		
Step	**Test data**	**Expected results**
Send GET request to /settings/gdpr/1	user_id = 1	System queries database
System retrieves settings	-	Settings found for user_id=1
Verify response	-	Response code: 200 OK JSON contains: - marketingCommunications (boolean) - airQualityDataCollection (boolean)

TC ID: TC-006 **Test idea:** Verify user can update GDPR settings **Preconditions:** - User is logged in (user_id=1) - GDPR settings exist		
Step	**Test data**	**Expected results**
Send POST request to /settings/gdpr/update	{"userId":1,"marketingCommunications":false,"timestamp":1706000000000}	System receives update request
System validates data	-	Data is valid
System updates database	-	marketingCommunications set to false

		Consent date recorded
Verify response	-	Response code: 200 OK Settings updated in database

1.4 <Generate Route Report (User)>

TC ID: TC-007
Test idea: Verify report generation for completed route
Preconditions:
- User is logged in (user_id=1)
- Active route exists (route_id=10)
- Route has 50+ waypoints with pollution data

Step	Test data	Expected results
Send POST request to /routes/stop_tracking	{"userId": 1, "routeId": "10"}	System receives request
System retrieves route and waypoints	-	Route found, waypoints retrieved
System calculates distance	-	Distance calculated using Haversine formula
System calculates duration and speed	-	Duration: end_time - start_time Speed: distance/time
System calculates pollution summary	-	Average and peak values for PM2.5, PM10, AQI
System creates report in database	-	Report saved with all statistics
Verify response	-	Response code: 200 OK JSON contains: {"message": "Tracking stopped..."} Report exists in route_reports table Pollution summary in route_pollution_summary

1.5 <Report Inaccurate Sensor (User)>

TC ID: TC-008
Test idea: Verify user can report inaccurate sensor
Preconditions:
- User is logged in
- Sensor with ID=5 exists

Step	Test data	Expected results
Send POST request to /report/sensors/5	sensor_id=5, report="Sensor showing incorrect PM2.5 values"	System receives report
System verifies sensor existence	-	Sensor ID=5 found in database

System creates report	-	Report created with status "pending"
Verify response	-	Response code: 200 OK Report saved in sensor_reports table

1.6 <Search Route History (User)>

TC ID: TC-009
Test idea: Verify user can retrieve route history list
Preconditions:
- User is logged in
- Completed routes exist in system

Step	Test data	Expected results
Send GET request to /history/routes	-	System queries database
System retrieves all completed routes	-	Routes with status="completed" retrieved
System calculates summary data	-	Duration, PM2.5 avg/max calculated for each
Verify response	-	Response code: 200 OK JSON array with objects containing: - id, date, status, time, duration - points, avgPm25, maxPm25

TC ID: TC-010
Test idea: Verify user can retrieve detailed route report
Preconditions:
- User is logged in
- Route with ID=10 exists with report

Step	Test data	Expected results
Send GET request to /details/10	route_id = 10	System queries database
System retrieves route and report	-	Route and pollution data retrieved
System calculates exposure level	-	Exposure level based on PM2.5 average: - Low: ≤12 - Moderate: 12–35 - High: 35–55 - Very High: >55
System generates recommendations	-	Health recommendations based on level
Verify response	-	Response code: 200 OK

		JSON contains complete route data plus: - exposureLevel - exposureAssessment - healthRecommendations array

1.7 <Registration In System (Guest)>

TC ID: TC-011
Test idea: Verify successful user registration
Preconditions:
- Email john.doe@example.com does not exist in system

Step	Test data	Expected results
Send POST request to /auth/register	{"username":"JohnDoe","email":"john.doe@example.com","password":"SecurePassword123"}	System receives registration data
System validates data	-	All fields present and valid
System checks email uniqueness	-	Email not found in database
System checks password length	-	Password ≤72 bytes (bcrypt limit)
System hashes password	-	Password hashed with bcrypt
System creates account	-	Account created with role "user"
System creates account settings	-	Default GDPR settings created
System generates JWT token	-	Token generated
Verify response	-	Response code: 200 OK JSON contains: accessToken, userId Account exists in database

TC ID: TC-012
Test idea: Verify registration rejection with existing email
Preconditions:
- Email existing@example.com already exists

Step	Test data	Expected results
Send POST request to /auth/register	{"username":"JaneDoe","email":"existing@example.com","password":"Password123"}	System receives registration data
System checks email uniqueness	-	Email found in database

Verify error response	-	Response code: 409 Conflict JSON contains: {"detail": "Account with this email already exists."}

1.8 <Stations Management (Admin)>

TC ID: TC-013
Test idea: Verify admin can list all stations
Preconditions:
- Admin is logged in
- Stations exist in the database

Step	Test data	Expected results
Send GET request to /admin/stations	-	System retrieves all stations
System calls get_all_stations()	-	All stations returned from database
Verify response	-	Response code: 200 OK JSON array with station objects containing: id, name, status, type, latitude, longitude, created

TC ID: TC-014
Test idea: Verify admin can view station measurements
Preconditions:
- Admin is logged in
- Station with ID=5 exists
- Station has sensor readings

Step	Test data	Expected results
Send GET request to /admin/stations/measurements/5	stationId = 5	System validates stationId
System retrieves measurements	-	Latest sensor readings retrieved
Verify response	-	Response code: 200 OK JSON array with: sensorId, date, time, PM25, PM10, AQI, unit

TC ID: TC-015
Test idea: Verify admin can change station status
Preconditions:
- Admin is logged in
- Station with ID=5 exists (is_active=true)

Step	Test data	Expected results

Send POST request to /admin/stations/5/status	body: false	System validates stationId
System calls update_station_status(5, false)	-	Station is_active updated to false
Verify response	-	Response code: 200 OK JSON: {"ok": true}

TC ID: TC-016
Test idea: Verify error when viewing measurements for non-existent station
Preconditions:
- Admin is logged in
- Station with ID=99999 does not exist

Step	**Test data**	**Expected results**
Send GET request to /admin/stations/measurements/99999	stationId = 99999	System checks station existence
Station not found	-	Returns error
Verify error response	-	Response code: 404 Not Found JSON: {"detail": "Station not found"}

2. Testy systemowe

File: test_api_marek_gagaczowski.py
Number of tests: 4

- **UC2:** Login with valid credentials
- **UC2:** Login with invalid password
- **UC2:** Login with non-existent email
- **UC1:** Get route info - route not found

File: test_api_blazej_kowal.py
Number of tests: 5

- **UC3:** Get GDPR settings
- **UC3:** Update GDPR settings
- **UC3:** Update GDPR settings - no fields provided
- **UC4:** Stop tracking (generate route report)
- **UC4:** Stop tracking - no active route

File: test_api_michal_strugarek.py
Number of tests: 5

- **UC6:** Get route history
- **UC6:** Get route details

- **UC6:** Get route details - route not found
  UC5: Report inaccurate sensor
- **UC5:** Report sensor - sensor not found

File: test_api_alina_lenart.py
Number of tests: 10

- **UC7:** Register with valid data
- **UC7:** Register with existing email
- **UC7:** Register with empty username
- **UC7:** Register with short password
- **UC7:** Register with too long password
- **UC7:** Register with missing fields
- **UC8:** List all stations
- **UC8:** List stations - empty list
- **UC8:** Get station measurements
- **UC8:** Get measurements - station not found
- **UC8:** Get measurements - invalid station ID
- **UC8:** Get measurements - empty measurements list
- **UC8:** Change station status (deactivate)
- **UC8:** Change station status (activate)
- **UC8:** Change station status - station not found
- **UC8:** Change station status - invalid station ID

How the automation was implemented (tools and approach)

- **Test framework:** pytest
- **API test client:** fastapi.testclient.TestClient

The tests were executed automatically by **pytest**. API requests were performed using TestClient, which simulates HTTP calls to the FastAPI application without requiring a deployed server.

Run with:
cd tests
pytest -v

Example screenshot of passed tests:

```
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_valid_data PASSED
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_existing_email PASSED
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_empty_username PASSED
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_empty_email PASSED
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_empty_password PASSED
functional/test_api_alina_lenart.py::TestRegistrationAPI::test_register_with_too_long_password PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_list_all_stations PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_list_stations_empty PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_get_station_measurements PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_get_station_measurements_not_found PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_get_station_measurements_invalid_id PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_get_station_measurements_empty_list PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_change_station_status PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_change_station_status_activate PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_change_station_status_not_found PASSED
functional/test_api_alina_lenart.py::TestAdminStationsAPI::test_change_station_status_invalid_id PASSED
functional/test_api_blazej_kowal.py::TestBlazejKowalAPI::test_get_gdpr_settings_success PASSED
functional/test_api_blazej_kowal.py::TestBlazejKowalAPI::test_get_gdpr_settings_not_found PASSED
functional/test_api_blazej_kowal.py::TestBlazejKowalAPI::test_update_gdpr_settings_success PASSED
functional/test_api_blazej_kowal.py::TestBlazejKowalAPI::test_update_gdpr_settings_no_fields PASSED
functional/test_api_blazej_kowal.py::TestBlazejKowalAPI::test_stop_tracking_generates_report PASSED
functional/test_api_marek_gagaczowski.py::TestMarekGagaczowskiAPI::test_login_with_valid_credentials PASSED
functional/test_api_marek_gagaczowski.py::TestMarekGagaczowskiAPI::test_login_with_invalid_password PASSED
functional/test_api_marek_gagaczowski.py::TestMarekGagaczowskiAPI::test_login_with_nonexistent_email PASSED
functional/test_api_marek_gagaczowski.py::TestMarekGagaczowskiAPI::test_get_route_info_not_found PASSED
functional/test_api_michal_strugarek.py::TestMichalStrugarekAPI::test_get_route_history_success PASSED
functional/test_api_michal_strugarek.py::TestMichalStrugarekAPI::test_get_route_details_success PASSED
functional/test_api_michal_strugarek.py::TestMichalStrugarekAPI::test_get_route_details_not_found PASSED
functional/test_api_michal_strugarek.py::TestMichalStrugarekAPI::test_report_sensor_endpoint_format PASSED
functional/test_api_michal_strugarek.py::TestMichalStrugarekAPI::test_report_sensor_invalid_format PASSED
```

3. Testy jednostkowe i mapowanie przypadków użycia do logiki biznesowej

1. Marek Gagaczowski

UC: Monitor Air Quality (User)

- **Endpoint:** GET /routes/{route_id}
- **File:** backend/routers/routes.py:153
- **Business Logic Methods:**
  - select_measurement_stations(waypoints) - backend/algorithms/route_station_selector.py:5
  - get_latest_readings_for_station(station_id) - backend/models/sensors.py:112
  - calculate_alert(pm25, pm10, aqi) - backend/algorithms/alert.py:1
  - haversine(lat1, lon1, lat2, lon2) - backend/algorithms/util.py:3

UC: Log Into Application (Guest)

- **Endpoint:** POST /auth/login
- **File:** backend/routers/users.py:47
- **Business Logic Methods:**
  - get_account_by_email(email) - backend/models/accounts.py:29
  - create_access_token(data, expires_delta) - backend/routers/users.py:39

2. Błażej Kowal

UC: Manage GDPR Settings (User)

- **Endpoints:**
  - GET /settings/gdpr/{user_id}
  - POST /settings/gdpr/update

- **Files:** backend/routers/gdpr.py:21, backend/routers/gdpr.py:32
- **Business Logic Methods:**
  - get_account_settings(account_id) - backend/models/accounts.py:83
  - update_account_settings(account_id, data) - backend/models/accounts.py:92

UC: Generate Route Report (User)

- **Endpoint:** POST /routes/stop_tracking
- **File:** backend/routers/routes.py:74
- **Business Logic Methods:**
  - calculate_route_distance(waypoints) - backend/algorithms/report.py:4
  - count_route_waypoints(waypoints) - backend/algorithms/report.py:18
  - calculate_pollution_summary(waypoints) - backend/algorithms/report.py:21

3. Michał Strugarek

UC: Report Inaccurate Sensor (User)

- **Endpoint:** POST /report/sensors/{sensor_id}
- **File:** backend/routers/reports.py:33
- **Business Logic Methods:**
  - get_sensor_by_id(sensor_id) - backend/models/sensors.py:20
  - create_sensor_report(sensor_id, account_id, description) - backend/models/sensor_reports.py:83
  - get_nearest_stations(latitude, longitude, limit) - backend/models/sensor_reports.py:10

UC: Search Route History (User)

- **Endpoints:**
  - GET /history/routes
  - GET /details/{route_id}
- **Files:** backend/routers/history.py:73, backend/routers/history.py:119
- **Business Logic Methods:**
  - format_duration(seconds) - backend/routers/history.py:36
  - get_exposure_level(avg_pm25) - backend/routers/history.py:42
  - get_exposure_assessment(level) - backend/routers/history.py:54
  - get_health_recommendations(level) - backend/routers/history.py:64

4. Alina Lenart

UC: Registration in the System (Guest)

- **Endpoint:** POST /auth/register
- **File:** backend/routers/users.py:57
- **Business Logic Methods:**

- get_account_by_email(email) - backend/models/accounts.py:29
- create_account(data) - backend/models/accounts.py:3
- create_account_settings(data) - backend/models/accounts.py:66
- create_access_token(data, expires_delta)- backend/routers/users.py:39

UC: Admin - Stations Management

- **Endpoints:**
  - GET /admin/stations
  - GET /admin/stations/measurements/{stationId}
  - POST /admin/stations/{stationId}/status
- **Files:** backend/routers/admin.py:13, backend/routers/admin.py:18, backend/routers/admin.py:31
- **Business Logic Methods:**
  - get_all_stations() - backend/models/stations.py:4
  - get_station_measurements(station_id) - backend/models/stations.py:129
  - update_station_status(station_id, is_active) - backend/models/stations.py:106

Methods

- **Marek Gagaczowski**
  - select_measurement_stations(waypoints) - algorithms/route_station_selector.py:5
    UC1: Monitor Air Quality - 3 tests
  - calculate_alert(pm25, pm10, aqi) - algorithms/alert.py:1
    UC1: Monitor Air Quality - 6 tests

- **Błażej Kowal**
  - calculate_route_distance(waypoints) - algorithms/report.py:4
    UC4: Generate Route Report - 4 tests
  - calculate_pollution_summary(waypoints) - algorithms/report.py:21
    UC4: Generate Route Report - 4 tests

- **Michał Strugarek**
  - format_duration(seconds) - routers/history.py:36
    UC6: Search Route History - 5 tests
  - get_exposure_level(avg_pm25) - routers/history.py:42
    UC6: Search Route History - 8 tests

- **Alina Lenart**
  - create_access_token(data, expires_delta) - routers/users.py:39
    UC7: Registration in the System - 4 tests
  - haversine(lat1, lon1, lat2, lon2) - algorithms/util.py:3
    UC7/UC8: shared utility (station distance calculations) - 5 tests

Total: 8 methods, **39 unit tests**

Sample methods:

Sample 1: calculate_alert - typical and edge case (Marek Gagaczowski)

```
def test_calculate_alert_low(self):
    result = calculate_alert(10, 20, 50)
    assert result == "Low"

def test_calculate_alert_high(self):
    result = calculate_alert(80, 110, 160)
    assert result == "High"

def test_calculate_alert_none_values(self):
    result = calculate_alert(None, None, None)
    assert result == "Low"
```

Sample 2: calculate_route_distance - typical and edge case (Błażej Kowal)

```
def test_calculate_route_distance_typical(self):
    waypoints = [
        {"latitude": 50.0, "longitude": 19.0},
        {"latitude": 50.01, "longitude": 19.01},
        {"latitude": 50.02, "longitude": 19.02}
    ]
    result = calculate_route_distance(waypoints)
    assert result > 0
    assert isinstance(result, float)

def test_calculate_route_distance_empty_waypoints(self):
    result = calculate_route_distance([])
    assert result == 0.0
```

Sample 3: get_exposure_level - typical and edge case (Michał Strugarek)

```
def test_get_exposure_level_low(self):
    result = get_exposure_level(10.0)
    assert result == "Low"

def test_get_exposure_level_very_high(self):
    result = get_exposure_level(60.0)
    assert result == "Very High"

def test_get_exposure_level_none(self):
    result = get_exposure_level(None)
    assert result == "Unknown"
```

Sample 4: create_access_token - typical and expiration (Alina Lenart)

```
def test_create_access_token_typical(self):
    data = {"sub": "123", "email": "test@example.com"}
    token = create_access_token(data, expires_delta=60)
    assert token is not None
    assert isinstance(token, str)
    decoded = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    assert decoded["sub"] == "123"
    assert decoded["email"] == "test@example.com"

def test_create_access_token_empty_data(self):
    token = create_access_token({})
    assert token is not None
    decoded = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    assert "exp" in decoded
```

Test coverage

Student	Methods Tested	Test Count	Pass Rate	Coverage
Marek Gagaczowski	2	10 tests	100%	100%
Błażej Kowal	2	8 tests	100%	100%
Michał Strugarek	2	12 tests	100%	100%
Alina Lenart	2	9 tests	100%	100%
TOTAL	**8 methods**	**39 tests**	**100%**	**100%**

4. Code quality rating

Complexity Statistics

Metric	Value	Unit
Total Functions Analyzed	97	functions
Average Cyclomatic Complexity	2.88	points

Functions with Grade A (1-5)	88	90.3%
Functions with Grade B (6-10)	4	4.0%
Functions with Grade C (11-20)	3	4.0%
Functions with Grade D (21-30)	1	0.8%
Functions with Grade F (30+)	1	0.8%

High Complexity Functions (C or above)

Function	File	Complexity	Grade
select_measurement_stations	algorithms/route_station_selector.py	21	D
update_gdpr_settings	routers/gdpr.py	18	C
get_route_details	routers/history.py	16	C
calculate_alert	algorithms/alert.py	13	C
get_route_info	routers/routes.py	12	C

Maintainability Index Statistics

Metric	Average	Min	Max	Unit
Maintainability Index (all files)	39.76	30.88	44.26	points

Function	File	Complexity	Grade
select_measurement_stations	algorithms/route_station_selector.py	21	D
update_gdpr_settings	routers/gdpr.py	18	C
get_route_details	routers/history.py	16	C
calculate_alert	algorithms/alert.py	13	C
get_route_info	routers/routes.py	12	C

Test Coverage Statistics

Layer	Avg Coverage	Min	Max
Algorithms	96%	83%	100%
Routers	77%	48%	100%
Models	12%	9%	20%
Other (main/db/scripts)	46%	0%	91%
Overall	54%	0%	100%

Coverage by Critical Modules

Module	Coverage	Layer
algorithms/alert.py	100%	Algorithms
routers/users.py	100%	Routers
routers/reports.py	48%	Routers
models/routes.py	9%	Models
generate_sample_data.py	0%	Other

Summary Metrics

Metric	Value	Grade	Status
Cyclomatic Complexity (avg)	2.83	A	Good
Maintainability Index (avg)	39.76	A	Good
Overall Test Coverage	54%	-	Moderate
Algorithms Coverage	96%	-	Excellent
Routers Coverage	77%	-	Good
Models Coverage	12%	-	Weak

The code quality of the EcoRoute backend (Python, FastAPI) was checked using radon and pytest-cov, looking at cyclomatic complexity and the Maintainability Index.

On average, the code is not very complex, with a complexity score of 2.88 (grade A). Most functions (>90%) are simple, but a few are more complicated, like select_measurement_stations (21), update_gdpr_settings (18), and get_route_details (16). These functions have lots of conditions and logic, so they might need some refactoring to make them easier to read and test.

```
backend/main.py
    F 12:0 root - A (1)

97 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.88659793814433)
PS C:\Users\Biossphere Gaming\Downloads\claude>
```

The Maintainability Index is also good (all files grade A), which means the code is pretty well organized and not too hard to maintain. Even the lowest scores are still fine.

Test coverage is 54%. The algorithms and main API endpoints are mostly covered, but the data models have very few tests.

Overall, the backend code is in good shape, with low complexity and decent maintainability. The main things to improve are refactoring the more complex functions and adding tests for the data layer to boost coverage.